

Auditoria de Segurança — RastrIA

- **Repositório:** Dom1ng0s/RastrIA
- **Commit auditado:** 1317ff0 (branch main)
- **Data:** 2026-08-30
- **Escopo:** código-fonte no repositório (backend Django/DRF, frontend React/Vite), dependências, configuração de deploy. Não inclui pentest dinâmico nem o código de backend descrito em agents/claude.md que ainda não foi commitado.

1. Reconhecimento

Camada	Stack	Gerenciador
Backend	Python 3.13, Django 5.x, Django REST Framework, djangorestframework-simplejwt, django-cors-headers, Gunicorn, WhiteNoise	pip / requirements.txt (sem lockfile)
Frontend	React 18, Vite 5, React Router 6, Axios, TanStack Query, Zustand, Zod, jsPDF, Tailwind	npm / package-lock.json
Deploy	Railway (Nixpacks). Frontend servido por serve -s. Backend via gunicorn rastria.wsgi + manage.py migrate	railway.json (raiz, backend/, frontend/)
Banco	dj-database-url → Postgres em produção, SQLite local	—

Estado atual (de agents/claude.md): o backend **ainda não está deployado** e o login do frontend é um seletor de papel mockado, sem chamada de API. As falhas de backend abaixo **não são exploráveis hoje em produção**, mas são **bloqueantes para o deploy** — o código, como está, fica criticamente vulnerável no momento em que o serviço subir.

2. Varredura de dependências

Backend — pip-audit -r backend/requirements.txt

No known vulnerabilities found (as faixas de versão resolvem para as últimas versões, todas sem CVE conhecida). Ver achado BAIXO-13 sobre ausência de lockfile.

Frontend — npm audit (4 vulnerabilidades: 1 alta, 3 moderadas)

Pacote	Versão	Severidade	Advisory	Correção
--------	--------	------------	----------	----------

vite	5.4.21	Alta	GHSA-fx2h-pf6j-xcff (bypass de server.fs.deny no Windows, CVSS 7.5)	vite@8.2.2 (major)
vite	5.4.21	Moderada	GHSA-4w7w-66w2-5vf9 (path traversal em .map), GHSA-v6wh-96g9-6wx3 (NTLMv2 hash via UNC / launch-editor)	vite@8.2.2
esbuild	0.21.5	Moderada	GHSA-67mh-4wv8-2f99 (dev server responde a qualquer origem)	via vite@8
react-router react-router-dom	/ 6.30.6	Moderada	GHSA-wrjc-x8rr-h8h6 (open redirect via backslash), GHSA-337j-9hxr-rhxg (constructor injection na hidratação SSR)	react-router-dom@7.18. 3 (major)

vite/esbuild afetam apenas o **servidor de desenvolvimento**. react-router é dependência de runtime.

3. SAST — código-fonte

Backend — o achado central. Todos os ViewSet do DRF seguem o mesmo padrão:

```
class XViewSet(viewsets.ModelViewSet):
    queryset = X.objects.all()          # sem escopo por usuário/instituição
    serializer_class = XSerializer
    permission_classes = [IsAuthenticated] # sem checagem de papel, sem has_object_permission
```

Não há `get_queryset()`, `perform_create()`, `has_object_permission()`, `throttling` ou `filter_backends` em lugar nenhum (grep confirma). A classe `TemPapel` existe em `apps/usuarios/permissions.py` mas **não é usada por nenhuma view**. Resultado: qualquer usuário autenticado tem CRUD completo sobre os dados de todos os outros — incluindo registros de saúde (dado sensível, LGPD Art. 11) e o campo `papel` (autorização).

Frontend — limpo nos pontos checados. Sem `dangerouslySetInnerHTML`, `eval`, `innerHTML`, `window.open`. A exportação CSV neutraliza injeção de fórmula (`escaparCelulaCsv`). O PDF é desenhado em vetor, sem `jsPDF.html()`. `RotaProtegida` é explicitamente só camada de apresentação.

4. Segredos e configuração

- **Nenhum segredo hardcoded** no código ou no histórico de commits recente. `.env` nunca foi commitado (`git log --all -- '*.env'` vazio). `.env.example` só contém placeholders.

- SECRET_KEY tem **fallback inseguro** em settings/base.py (ver ALTO-4).
- settings/dev.py: DEBUG=True, ALLOWED_HOSTS=[*], CORS_ALLOW_ALL_ORIGINS=True (ver BAIXO-11).
- settings/prod.py: define SECURE_SSL_REDIRECT, SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE, SECURE_PROXY_SSL_HEADER — mas **falta HSTS e CSRF_TRUSTED_ORIGINS** (ver MÉDIO-7).
- Cookies: a API é stateless (JWT), mas o Django admin usa sessão — flags Secure estão corretas em prod.

5. Infraestrutura e CI/CD

- **Sem Dockerfile** — build via Nixpacks (Railway). Sem análise de imagem aplicável.
- **Sem .github/workflows** — nenhum CI de segurança (SAST, npm audit, pip-audit, secret scanning, Dependabot). Ver BAIXO-14.
- railway.json do backend roda python manage.py migrate settings=rastria.settings.prod no start — aceitável, mas migração no boot de cada réplica pode causar corrida em escala.

6. Consolidação — achados e status

O repositório **já tinha triagem de segurança** antes desta auditoria (issues #59, #60, #63, #64, #65). Os achados abaixo foram cruzados com essas issues; só os não cobertos geraram issue nova (#103–#111, criadas em 2026-08-30).

#	Severidade	Vulnerabilidade	Local	Status / Issue
1	Crítico	IDOR / BOLA em registros de saúde qualquer usuário autenticado lê/edita/apaga dados de saúde de todos	backend/apps/saude/vie ws.py:14	Já rastreado — #59
2	Crítico → Alto	Escalonamento de privilégio: campo papel gravável no serializer + queryset global	backend/apps/usuarios/s erializers.py:9, views.py:16	#103 (aberta) — complementa #59/#67
3	Alto	Ausência de escopo/autorização por papel nos demais ViewSets (instituições, vínculos, profissionais, solicitações, atendimentos, regras clínicas)	backend/apps/*/views.py	Já rastreado — #59 + #60

4	Alto	SECRET_KEY com backend/rastria/settings/ fallback inseguro e sem base.py:11 validação de presença em produção (repo público)	Já rastreado — #64
5	Médio	Sem rate limiting / backend/rastria/settings/ throttling no endpoint de base.py, urls.py:7 login JWT (brute force / password spraying)	#104 (aberta)
6	Médio	Refresh token JWT (7 dias) não revogável; base.py (SIMPLE_JWT) sem token_blacklist; logout só client-side	#105 (aberta)
7	Médio	Cabeçalhos de backend/rastria/settings/ segurança ausentes em prod.py produção: HSTS, CSRF_TRUSTED_ORIGINS, SECURE_REFERRER_POLICY	Já rastreado — #63
8	Médio	Dependências de build frontend/package.json do frontend vulneráveis: vite@5.4.21 (GHSA-fx2h-pf6j-xcff, alta), esbuild@0.21.5	#106 (aberta)
9	Médio	react-router@6.30.6 — frontend/package.json open redirect (GHSA-wrjc-x8rr-h8h6) e constructor injection (GHSA-337j-9hxr-rhxg)	#107 (aberta)
10	Baixo	Tokens JWT em frontend/src/lib/authToken localStorage (roubo via ns.js XSS) — decisão consciente/aceita	Já rastreado — #65 (closed, com plano de migração)
11	Baixo	settings/dev.py permissivo + backend/rastria/settings/ dev.py, manage.py:9 manage.py/.env.example e default para settings.dev	#108 (aberta)
12	Baixo	Django admin (/admin/) backend/rastria/urls.py:6 sem 2FA, rate limiting ou restrição de rede	#109 (aberta)
13	Baixo	requirements.txt sem backend/requirements.txt lockfile nem hash t pinning (build não reproduzível / supply chain)	#110 (aberta)

14	Baixo	Sem CI de segurança repo (Dependabot, npm audit/pip-audit, CodeQL) no repositório	Já rastreado — #68 (comentário adicionado com o escopo de dep-scan)
15	Baixo	Frontend servido por package.json, serve sem CSP / railway.json X-Content-Type-Options / X-Frame-Options / Referrer-Policy	#111 (aberta)

7. Detalhamento e recomendações

[CRÍTICO-1] IDOR / Broken Object-Level Authorization em RegistroSaude

Problema. RegistroSaudeViewSet (backend/apps/saude/views.py:14) expõe RegistroSaude.objects.all() com permission_classes = [IsAuthenticated] e sem get_queryset(). As rotas GET/POST/PUT/PATCH/DELETE /api/registros-saude/ e /api/registros-saude/{id}/ ficam disponíveis para qualquer usuário autenticado, sobre os registros de **todos** os usuários. O mesmo vale para RegraVerificacaoViewSet no mesmo arquivo.

Localização. backend/apps/saude/views.py:5-16.

Impacto. Vazamento em massa de dado de saúde — categoria de **dado pessoal sensível** (LGPD Art. 11): exames laboratoriais, avaliação cardiológica, bioimpedância, desempenho físico, associados a um usuário identificável. Um integrante lê o histórico clínico de colegas; qualquer conta lê o de qualquer pessoa. Também permite adulteração e exclusão de registros clínicos (integridade de prontuário). Incidente reportável à ANPD.

Recomendação.

- Implementar get_queryset() restringindo a RegistroSaude.objects.filter(usuario=self.request.user), com exceção explícita e auditável para o profissional vinculado (VinculoCuidado ativo) e o médico do trabalho da instituição.
- Forçar usuario no perform_create() a partir de request.user (hoje o serializer aceita usuario do corpo — permite gravar em nome de terceiros).
- Adicionar has_object_permission() para PUT/PATCH/DELETE.
- Cobrir com testes de autorização (usuário A não acessa registro de B).

Referência. OWASP API1:2023 (Broken Object Level Authorization); OWASP A01:2021; CWE-639; LGPD Art. 11 e 46.

[CRÍTICO-2] Escalonamento de privilégio via campo papel gravável

Problema. UsuarioSerializer (backend/apps/usuarios/serializers.py:9) inclui papel na lista fields sem marcá-lo como read_only. UsuarioViewSet (views.py:16) usa Usuario.objects.all() com só IsAuthenticated

e sem `get_queryset()`. Um usuário autenticado envia `PATCH /api/usuarios/{qualquer_id}/` com `{"apel": "gerente"}` e passa a ter o papel de Gerente/Comandante (ou Médico) — que é justamente o eixo de autorização do produto.

Localização. `backend/apps/usuarios/serializers.py:1-12;` `backend/apps/usuarios/views.py:6-16` (a docstring da própria view reconhece o `TODO`).

Impacto. Bypass total da segregação por papel: acesso a painéis e dados de gerência/médicos, listagem de PII de todos os usuários (email, nome), alteração de dados de qualquer conta. Combinado com CRÍTICO-1 e ALTO-3, um único usuário comum compromete todo o sistema.

Recomendação.

- `read_only_fields = ["id", "apel"]` no serializer; mudança de papel só por endpoint administrativo dedicado protegido por `EhGerente/IsAdminUser`.
- `get_queryset()` restringindo a lista/detalhe ao próprio usuário, exceto staff.
- Desabilitar `create/destroy` no `UsuarioViewSet` (contas nascem por provisionamento institucional — ver `agents/claude.md`); usar `RetrieveUpdateAPIView` só para `/me`.
- Nunca expor `is_staff`, `is_superuser`, `user_permissions`, `groups` no serializer.

Referência. OWASP API3:2023 (Broken Object Property Level Authorization / Mass Assignment); OWASP A01:2021; CWE-915; CWE-269.

[ALTO-3] Ausência de escopo de `queryset` e autorização por papel nos demais `ViewSets`

Problema. `InstituicaoViewSet`, `VinculoInstituicaoViewSet`, `ProfissionalViewSet`, `SolicitacaoViewSet`, `AtendimentoViewSet`, `VinculoCuidadoViewSet` e `RegraVerificacaoViewSet` repetem o padrão `objects.all()` + `IsAuthenticated`, com CRUD completo.

Localização. `backend/apps/instituicoes/views.py,` `backend/apps/profissionais/views.py,` `backend/apps/atendimentos/views.py,` `backend/apps/saude/views.py.`

Impacto por endpoint.

- `POST /api/vinculos-institucionais/` com `{"usuario": <eu>, "instituicao": <x>, "apel": "gerente"}` → **escalonamento de privilégio institucional** (o papel do vínculo é gravável).
- `PATCH /api/regras-verificacao/{id}/` → adulteração das **faixas de referência clínica** usadas na verificação automática de saúde; um valor perigoso passa a ser reportado como "normal". Impacto de segurança do paciente.
- `PATCH /api/solicitacoes/{id}/` com `{"status": "confirmada"}` → pula a confirmação do profissional. A docstring da view alerta que isso não deve ser possível; além do problema de autorização, há risco de enquadramento regulatório (Parecer CFM nº 15/2026, citado em `agents/claude.md`).
- `GET /api/profissionais/, /api/instituicoes/, /api/atendimentos/, /api/vinculos-cuidado/` → enumeração de toda a base; `PATCH/DELETE` de registros de terceiros.

Recomendação.

- Aplicar as subclasses de `TemPapel` já existentes (`EhGerente`, `EhMedicoOuMedicoDoTrabalho`, `EhEducadorFisico`) como `permission_classes` por view.
- `get_queryset()` escopado por `request.user` e pela instituição do usuário (via `VinculoInstituicao`).
- `RegraVerificacao`: leitura para autenticados, escrita só para `IsAdminUser`.
- `Solicitacao`: remover PATCH genérico; expor ações explícitas `@action` confirmar/recusar restritas ao profissional destinatário. Campos como `status` como `read_only` no serializer.
- Testes de autorização para cada papel × cada endpoint.

Referência. OWASP API1:2023, API3:2023, API5:2023 (Broken Function Level Authorization); OWASP A01:2021.

[ALTO-4] SECRET_KEY com fallback inseguro e sem validação em produção

Problema. `backend/rastria/settings/base.py:11`:

```
SECRET_KEY = os.getenv("SECRET_KEY", "inseguro-trocar-em-producao")
```

`settings/prod.py` importa de `base` e não valida que a variável foi definida. Se o serviço Railway do backend subir sem `SECRET_KEY` no ambiente, o Django roda com uma chave **presente no repositório público**.

Localização. `backend/rastria/settings/base.py:11`; `backend/rastria/settings/prod.py`.

Impacto. Com a `SECRET_KEY` conhecida: forja de cookies de sessão assinados (acesso ao Django admin), forja de tokens de reset de senha (`PasswordResetTokenGenerator`), forja de qualquer valor assinado por `django.core.signing`. Comprometimento total da autenticação de sessão.

Recomendação.

- Em `prod.py`, remover o default: `SECRET_KEY = os.environ["SECRET_KEY"]` (falha no boot se ausente), ou validação explícita com `ImproperlyConfigured`.
- Rotacionar a `SECRET_KEY` antes do primeiro deploy (a atual está no histórico do Git).
- Rodar `python manage.py check` deploy no CI e bloquear o merge em caso de warning.

Referência. OWASP A05:2021 (Security Misconfiguration); CWE-798; Django deployment checklist.

[MÉDIO-5] Sem rate limiting no login JWT

Problema. `REST_FRAMEWORK` em `base.py` não define `DEFAULT_THROTTLE_CLASSES/DEFAULT_THROTTLE_RATES`. `TokenObtainPairView` (`urls.py:7`) fica sem qualquer limite de tentativas.

Impacto. Brute force de senha e password spraying contra `/api/auth/token/` e contra `/admin/login/`. Agravado por MÉDIO-6 (sem `lockout`) e pela regra de senha de apenas 8 caracteres.

Recomendação. `ScopedRateThrottle` no endpoint de token (ex.: 5/min por IP); considerar `django-axes` para `lockout` por conta/IP; `throttle` global `AnonRateThrottle/UserRateThrottle`.

Referência. OWASP API4:2023 (Unrestricted Resource Consumption); OWASP A07:2021; CWE-307.

[MÉDIO-6] Refresh token não revogável; logout apenas client-side

Problema. `rest_framework_simplejwt.token_blacklist` não está em `INSTALLED_APPS`. `SIMPLE_JWT` não define `ROTATE_REFRESH_TOKENS` nem `BLACKLIST_AFTER_ROTATION` (apesar de `frontend/src/lib/api.js` comentar que a rotação está ligada). O refresh token vale 7 dias e não há como invalidá-lo.

Impacto. Logout, troca de senha ou perda de dispositivo não encerram a sessão no servidor — um refresh token vazado continua emitindo access tokens por até 7 dias. O `TODO` em `SessaoInativa.jsx` já registra a lacuna.

Recomendação. Adicionar `token_blacklist` (+ migração); `ROTATE_REFRESH_TOKENS=True`, `BLACKLIST_AFTER_ROTATION=True`; endpoint de logout que faz blacklist do refresh; blacklist de todos os tokens do usuário na troca de senha.

Referência. OWASP A07:2021; CWE-613 (Insufficient Session Expiration).

[MÉDIO-7] Cabeçalhos de segurança ausentes em produção

Problema. `settings/prod.py` cobre redirect HTTPS e cookies Secure, mas não define:

- `SECURE_HSTS_SECONDS` (+ `SECURE_HSTS_INCLUDE_SUBDOMAINS`, `SECURE_HSTS_PRELOAD`)
- `CSRF_TRUSTED_ORIGINS` (necessário no Django 4+ para o admin sob HTTPS em domínio próprio)
- `SECURE_REFERRER_POLICY` (default é `same-origin`; explicitar `strict-origin-when-cross-origin` ou mais restrito)

Impacto. Sem HSTS, a primeira requisição de cada cliente continua vulnerável a downgrade/SSL-strip mesmo com `SECURE_SSL_REDIRECT`. Sem `CSRF_TRUSTED_ORIGINS`, o login do admin pode quebrar ou ser mal configurado.

Recomendação. Definir `SECURE_HSTS_SECONDS = 31536000`, `SECURE_HSTS_INCLUDE_SUBDOMAINS = True`, `SECURE_HSTS_PRELOAD = True`, `CSRF_TRUSTED_ORIGINS = ["https://rastria.up.railway.app", ...]`. Rodar `manage.py check` [deploy](#).

Referência. OWASP A05:2021; OWASP Secure Headers Project; MDN HSTS.

[MÉDIO-8] Dependências de build do frontend vulneráveis (vite, esbuild)

Problema. `npm audit`: `vite@5.4.21` afetado por **GHSA-fx2h-pf6j-xcff** (alta, CVSS 7.5 — bypass de `server.fs.deny` em caminhos alternativos no Windows, leitura arbitrária de arquivo pelo dev server), `GHSA-4w7w-66w2-5vf9` (path traversal em `.map`), `GHSA-v6wh-96g9-6wx3` (divulgação de hash NTLMv2 via UNC). `esbuild@0.21.5`: `GHSA-67mh-4ww8-2f99` (o dev server aceita requisições de qualquer site e

devolve a resposta).

Impacto. Restrito ao **ambiente de desenvolvimento** (o dev server não roda em produção). Um site malicioso aberto no navegador do dev, ou um projeto malicioso, pode ler arquivos do disco. Sem impacto no bundle servido em produção.

Recomendação. Atualizar para vite@8.x (major — testar o build) e @vitejs/plugin-react compatível. Enquanto não atualiza: não abrir sites não confiáveis com o dev server rodando; fixar o host do dev server em 127.0.0.1.

Referência. GHSA-fx2h-pf6j-xcff, GHSA-67mh-4wv8-2f99, GHSA-4w7w-66w2-5vf9, GHSA-v6wh-96g9-6wx3.

[MÉDIO-9] react-router@6.30.6 — open redirect e constructor injection

Problema. npm audit: **GHSA-wrjc-x8rr-h8h6** (open redirect via backslash em <Link>/useNavigate — bypass do CVE-2025-68470) e **GHSA-337j-9hxr-rhxg** (CWE-470, injeção de construtor arbitrário via deserializeErrors() na hidratação SSR).

Impacto. Open redirect é relevante em fluxos de "voltar para a rota tentada após login" — RotaProtegida.jsx já guarda location.state.from, e a issue #61 prevê usar esse valor no redirect pós-login; um from com backslash pode redirecionar para domínio externo (phishing). O vetor SSR não se aplica (app é SPA sem SSR).

Recomendação. Atualizar para react-router-dom@7.18.3 (major — há guia de migração; a superfície usada no projeto é pequena). Enquanto não atualiza: ao implementar o redirect pós-login, validar que from começa com / e não com // nem \.

Referência. GHSA-wrjc-x8rr-h8h6, GHSA-337j-9hxr-rhxg; CWE-601.

[BAIXO-10] Tokens JWT em localStorage

Problema. frontend/src/lib/authTokens.js guarda access e refresh token em localStorage.

Impacto. Qualquer XSS no app exfiltra os dois tokens (o refresh vale 7 dias — ver MÉDIO-6). **Já é uma decisão consciente e documentada** (header do arquivo, issue #65), com plano de migrar o refresh para cookie httpOnly+Secure quando o backend estiver no ar. Mitigações parciais já existentes: access token de 1h, encerramento de sessão por inatividade de 20 min (issue #92).

Recomendação. Executar o plano da issue #65: refresh token em cookie httpOnly+Secure+SameSite=Strict, access token curto só em memória. Adicionar CSP (ver BAIXO-15) para reduzir a superfície de XSS. Manter rastreado até a migração.

Referência. OWASP A07:2021; OWASP "HTML5 Security Cheat Sheet — Local Storage"; CWE-922.

[BAIXO-11] settings/dev.py permissivo + default para dev

Problema. settings/dev.py: DEBUG=True, ALLOWED_HOSTS=["*"], CORS_ALLOW_ALL_ORIGINS=True. backend/manage.py:9 e backend/.env.example apontam DJANGO_SETTINGS_MODULE para rastria.settings.dev. Só wsgi.py/asgi.py e o railway.json do backend forçam prod.

Impacto. Risco de subir o serviço (ou rodar um comando de manutenção que toca a rede) com settings de dev: páginas de erro com stack trace e settings expostos, Host header irrestrito, CORS totalmente aberto (qualquer site chama a API com credenciais do usuário).

Recomendação. .env.example com DJANGO_SETTINGS_MODULE=rastria.settings.prod (dev sobrescreve localmente). Considerar não ter default em manage.py. manage.py check deploy no CI.

Referência. OWASP A05:2021; Django DEBUG deployment notes; CWE-1188.

[BAIXO-12] Django admin exposto sem proteção adicional

Problema. /admin/ registrado em backend/rastria/urls.py:6 sem rate limiting, 2FA ou restrição de rede.

Impacto. Superfície de brute force administrativo (ver MÉDIO-5) e alvo direto se a SECRET_KEY vazar (ALTO-4). O admin dá acesso irrestrito a todos os dados de saúde.

Recomendação. Mover para path não previsível; django-axes para logout; restringir por IP/VPN se viável; django-otp/django-two-factor-auth para contas de staff; ADMIN_URL via env.

Referência. OWASP A04:2021 (Insecure Design); OWASP A07:2021.

[BAIXO-13] requirements.txt sem lockfile nem hash pinning

Problema. backend/requirements.txt usa faixas (Django>=5.0,<6.0 etc.). Não há requirements.lock, pip-compile ou require-hashes.

Impacto. Build não reprodutível entre ambientes; exposto a uma versão futura comprometida de dependência ou transitiva (o upper bound não protege de patch malicioso). Nenhuma CVE nas versões atuais (pip-audit limpo).

Recomendação. Adotar pip-tools (requirements.in → requirements.txt com hashes) ou uv/Poetry com lockfile versionado. Rodar pip-audit no CI.

Referência. OWASP A06:2021 (Vulnerable and Outdated Components); OWASP A08:2021 (Software and Data Integrity Failures).

[BAIXO-14] Sem CI de segurança no repositório

Problema. Sem .github/workflows. Nenhuma verificação automática de dependências, SAST ou segredos

no PR.

Impacto. Regressões de segurança e dependências vulneráveis entram sem sinal (ex.: as 4 do npm audit atuais).

Recomendação. Habilitar Dependabot (.github/dependabot.yml para pip e npm); workflow com npm audit `audit-level=high, pip-audit, python manage.py check_deploy`; GitHub CodeQL (Python + JS); gitleaks/secret scanning.

Referência. OWASP A06:2021; OWASP SAMM — Verification.

[BAIXO-15] Frontend servido sem cabeçalhos de segurança

Problema. `serve -s frontend/dist -l $PORT` (pacote `serve`, em `package.json:8` e `railway.json`) não adiciona `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `X-Frame-Options/frame-ancestors`, `Referrer-Policy`.

Impacto. Sem CSP, um XSS não encontra nenhuma barreira de contenção (agrava BAIXO-10 — exfiltração de token). Sem `X-Frame-Options`, a app pode ser embutida em `iframe` (clickjacking).

Recomendação. Servir atrás de um proxy/edge (ou trocar `serve` por um servidor configurável) que injete: `Content-Security-Policy (default-src 'self'; connect-src 'self' <API>; frame-ancestors 'none')`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` mínimo. Adicionar `<meta http-equiv="Content-Security-Policy">` como reforço no `index.html`.

Referência. OWASP A05:2021; OWASP Secure Headers Project; MDN CSP.

8. Priorização recomendada

1. **Antes de qualquer deploy do backend:** CRÍTICO-1, CRÍTICO-2, ALTO-3, ALTO-4 (todos de autorização/config e triviais de explorar assim que a API subir).
2. **Junto do deploy:** MÉDIO-5, MÉDIO-6, MÉDIO-7, BAIXO-11, BAIXO-12.
3. **Backlog próximo:** MÉDIO-8, MÉDIO-9 (bump de major, precisa de teste), BAIXO-13, BAIXO-14, BAIXO-15.
4. **Rastrear até a migração planejada:** BAIXO-10 (issue #65).

9. Observações de conformidade (LGPD)

O produto trata **dados pessoais sensíveis de saúde** (LGPD Art. 11) de titulares identificáveis, em contexto institucional (B2G — Polícia Militar citada como piloto). CRÍTICO-1 e ALTO-3 representam risco direto de incidente de segurança reportável à ANPD (Art. 48) e de responsabilização (Art. 44, 46). Recomenda-se, além das correções: registro de operações (log de acesso a registro de saúde), política de retenção, e revisão jurídica das páginas de Termos/Política (já marcadas como rascunho no código).

10. Issues abertas nesta auditoria (2026-08-30)

Issue	Título	Labels
#103	Campo papel gravável via API permite auto-escalonamento de privilégio	security, backend, blocker
#104	Endpoint de login JWT sem rate limiting / throttling	security, backend, severity: medium
#105	Refresh token JWT não é revogável; logout apenas no cliente	security, backend, severity: medium
#106	vite 5.4.21 e esbuild 0.21.5 com vulnerabilidades conhecidas	security, frontend, dependencies, severity: medium
#107	react-router 6.30.6: open redirect e constructor injection	security, frontend, dependencies, severity: medium
#108	dev.py permissivo e DJANGO_SETTINGS_MODULE default para dev	security, backend, severity: low
#109	Django admin sem rate limiting, 2FA ou restrição de rede	security, backend, severity: low
#110	requirements.txt sem lockfile nem hash pinning	security, backend, chore, severity: low
#111	Frontend servido sem headers de segurança (CSP, X-Frame-Options)	security, frontend, deploy, severity: low

Já rastreados (sem issue nova): #59, #60, #63, #64, #65; comentário adicionado em #68.

Auditoria automatizada assistida. Não substitui pentest dinâmico nem revisão de código manual aprofundada do backend completo (parte ainda não commitada).